

No Evidence That TaperNorm Increases Forgetting After a Web-to-Code Domain Shift: A Pre-Specified Pilot Study

Daniel Shdeed
University of the People
DanielShdeed@my.uopeople.edu

October 2026

Abstract

TaperNorm replaces a pre-norm Transformer’s internal normalization with a gated map that acts like RMSNorm early in training and then becomes a fixed, calibrated linear scaling. Without per-token normalization, such a model might forget more after a shift in the training data. We tested this in a small pilot with design, endpoint and decision rules fixed before training. Paired 17.7M-parameter models with internal RMSNorm or TaperNorm (three seeds) were trained on 150M web tokens, then continued for 100M tokens on web text or Python code. The primary endpoint is a difference-in-differences D in held-out web cross-entropy; positive D means extra forgetting under TaperNorm. Switching to Python raised web cross-entropy by about 1.66 nats/token in both models. Mean D was -0.022 nats/token (seeds -0.020, -0.021, -0.026), below the pre-specified +0.015 bound, so the pre-specified decision is to stop: we find no evidence that TaperNorm increases persistent forgetting in this setting. In exploratory analyses, D was near zero for most of continuation (after a brief early dip) and became negative as the learning rate annealed, and the code-specific activation-scale shift that motivated the hypothesis was small in both models. Code and all evaluation records are released.

1 Introduction

Normalization layers such as LayerNorm (Ba et al., 2016) and RMSNorm (Zhang and Sennrich, 2019) are a standard part of Transformer language models. In the common pre-norm arrangement (Xiong et al., 2020), every attention and MLP sub-layer reads its input through a normalizer that rescales each token’s activation vector by that token’s own statistics. A long line of work has asked whether normalization is necessary at all (Zhang et al., 2019b; Brock et al., 2021; He and Hofmann, 2024; Zhu et al., 2025). TaperNorm (Kanavalau et al., 2026) is a recent proposal for Transformers; we follow version 1 of its arXiv preprint throughout. It acts exactly as RMSNorm (or LayerNorm) during an initial gate warm-up, calibrates a fixed scaling branch during that warm-up, and then cosine-decays a global gate until each internal normalizer becomes a sample-independent linear map. That map can be folded into the adjacent linear projections at inference. Kanavalau et al. (2026) report that this matches normalized baselines under identical setups, and they measure up to $1.22\times$ higher throughput in an inference microbenchmark.

This study looks at a property that the original evaluation did not target: behaviour under a later shift in the training distribution. A normalizer is invariant to the scale of its input. If continued training on a new domain changes activation magnitudes, an RMSNorm layer absorbs

part of that change automatically, whereas a tapered layer applies a scale calibrated on the earlier distribution. One can conjecture that the tapered network must then absorb the shift through weight updates, which might overwrite structure that supports the original domain and so increase catastrophic forgetting (McCloskey and Cohen, 1989; French, 1999; Kirkpatrick et al., 2017). This argument motivates the hypothesis; the experiment tests the hypothesis, not the mechanism.

Hypothesis. H1: after training switches from web text to Python code, internal TaperNorm (without the auxiliary scale loss; final RMSNorm retained) increases the persistent deterioration of held-out web cross-entropy, relative to continuing web training, compared with internal RMSNorm. *Persistent* is defined in Section 3.1.

We ran a deliberately small, inexpensive pilot to decide whether H1 deserves a larger study. Its design (Section 3) pairs the two conditions on initialization and data order and branches each condition’s trained prefix into a web continuation and a Python continuation from an identical saved state. The design also fixes a difference-in-differences endpoint and an ordered set of decision rules before any training. Our contributions are:

- a paired, pre-specified design for measuring how an architectural choice affects forgetting under a domain shift, whose primary contrast decomposes exactly into branch-level forgetting, a control-branch term, per-document effects and per-token-class effects;
- the pilot’s result. All validity checks pass and the pre-specified decision is *stop: small observed effect*. Mean $D = -0.0223$ nats/token, against a $+0.015$ bound for a meaningful excess;
- exploratory analyses, labelled as such, suggesting that the premise of H1 was only weakly present: the code-specific change in activation scale was small and similar in both architectures;
- code, frozen configuration, data hashes and all evaluation and training records, sufficient to recompute every number in this paper.

The study is small (17,718,784 parameters, three seeds, one domain pair) and should be read as a screening result, not as a general statement about TaperNorm (Section 7).

2 Related work

Normalization and its removal. LayerNorm (Ba et al., 2016) and RMSNorm (Zhang and Sennrich, 2019) normalize each token independently, and the pre-norm placement (Xiong et al., 2020) is now standard in decoder-only language models. Fixup (Zhang et al., 2019b) and NF Nets (Brock et al., 2021) train deep residual networks without normalization through initialization, scaling and clipping choices. For Transformers, He and Hofmann (2024) study which standard components of the block can be removed, and Zhu et al. (2025) replace normalization layers with an element-wise tanh map. TaperNorm (Kanavalau et al., 2026) instead removes internal normalization gradually during training. Its analysis attributes the essential role of normalization to the *output* normalizer, which anchors scale; this is why our Taper-minus condition keeps the final RMSNorm.

Normalization, weight decay and effective learning rate. When a normalizer makes a network’s output invariant to the scale of some weights, weight decay no longer regularizes those weights in the usual sense; it shrinks them and thereby raises their effective learning rate (van Laarhoven, 2017; Zhang et al., 2019a). Dohare et al. (2024) document a loss of plasticity in deep continual learning, and Lyle et al. (2024) connect normalization and effective learning rates to the

loss of plasticity under nonstationary training. These results bear on our exploratory scale analysis (Section 5.4).

Forgetting under continued training. Catastrophic forgetting (McCloskey and Cohen, 1989; French, 1999) is the loss of earlier competence when a network is trained on new data, and it has motivated methods such as elastic weight consolidation (Kirkpatrick et al., 2017). The training regime, including learning-rate decay, affects how much a network forgets (Mirzadeh et al., 2020); larger pre-trained models forget less (Ramasesh et al., 2022); and the choice of normalization layer matters in online continual learning (Pham et al., 2022). In language modelling, continued pre-training on a new domain improves in-domain performance (Gururangan et al., 2020), but it can degrade performance on the original distribution; Gupta et al. (2023) and Ibrahim et al. (2024) study how learning-rate re-warming, re-decaying and replay trade adaptation against this forgetting. Our design deliberately uses no replay, re-warming or other mitigation. It isolates how much forgetting the normalization choice itself causes, relative to a matched control branch.

Pre-specification. Fixing hypotheses, endpoints and analysis rules before seeing outcomes protects against flexible analysis (Nosek et al., 2018). Section 3.6 states exactly what was fixed, and when, including one disclosure about the timing of the public record.

3 Pre-specified design

3.1 Notation and estimands

We write u for the global optimizer update ($u = 1, \dots, 15,260$) and $s = u - 9,156$ for the continuation update within a branch. Two different events fall on the number 6,104: the TaperNorm gate reaches zero at global update $u = 6,104$, and the primary endpoint is continuation update $s = 6,104$ (global update $u = 15,260$). They coincide because the protocol ends the taper at 40% of the 15,260-update trajectory, which equals the continuation length. We always state whether a number is a u or an s .

Let $L(m, b, s)$ be the token-weighted cross-entropy on the held-out web development set (nats per supervised token, 2,097,152 labels), for normalization condition $m \in \{\text{R}, \text{T}\}$ (RMS, Taper-minus), after s continuation updates on branch $b \in \{\text{web}, \text{code}\}$. Let $L(m, \text{prefix})$ be the same quantity at the switch. The primary endpoint is

$$D(s) = [L(\text{T}, \text{code}, s) - L(\text{T}, \text{web}, s)] - [L(\text{R}, \text{code}, s) - L(\text{R}, \text{web}, s)], \quad (1)$$

evaluated at $s = 6,104$. Positive D means that, relative to its own web-continuation control, the Taper-minus model loses more web performance under the Python shift than the RMS model does. Within each condition the code-minus-web difference cancels everything the two branches share, including the condition-specific quality of the prefix and generic continued-training drift.

Persistent. We call a deterioration *persistent* if it is present at the fixed endpoint $s = 6,104$, after 100M continuation tokens and at the end of the learning-rate schedule. A difference visible only early in continuation ($s \leq 1,000$) is *transient*, which the decision rules treat separately (Table 1, rule 5).

We also report the actual forgetting $F(m, b) = L(m, b, 6104) - L(m, \text{prefix})$ and the change in the between-condition gap during web continuation, $G_{\text{web}} = [L(\text{T}, \text{prefix}) - L(\text{R}, \text{prefix})] -$

$[L(\text{T}, \text{web}, 6104) - L(\text{R}, \text{web}, 6104)]$. These satisfy the identity

$$D = \underbrace{F(\text{T}, \text{code}) - F(\text{R}, \text{code})}_Q + G_{\text{web}}, \quad (2)$$

so $Q = D - G_{\text{web}}$ is the direct difference in Python-branch forgetting. The identity is checked numerically in the analysis code. Because CE is token-weighted, D decomposes exactly into per-document terms D_i (weighted by each document’s scored-label count) and into four disjoint GPT-2 token classes: whitespace (W), text containing an alphanumeric character (A), other punctuation and symbols (P), and special, control or invalid-fragment tokens (X). A fifth, overlapping flag R marks rare tokens: those that occur fewer than 100 times in that seed’s prefix training stream.

3.2 Models and conditions

All models are pre-norm decoder-only Transformers adapted from nanoGPT (Karpathy, 2022):

- 6 layers, width 256, 4 heads of size 64, and an exact-GELU MLP of width 1,024;
- learned absolute positions, context 512, no biases and no dropout;
- GPT-2 token embeddings (Radford et al., 2019), tied to the output layer (vocabulary 50,257).

The RMS model has 17,718,784 parameters; Taper-minus adds 3,072 for its second gain vectors. About 73% of all parameters are in the token-embedding table. Both conditions keep the final RMSNorm.

Initialization. All linear and embedding weights (token and position) are drawn from $\mathcal{N}(0, 0.02^2)$. The two residual output projections of each block (the attention output projection and the MLP down-projection) use $\mathcal{N}(0, (0.02/\sqrt{12})^2)$, where $12 = 2 \times 6$ layers. All gains start at 1.

RMS. All 12 internal normalizers are RMSNorm with $\epsilon = 10^{-6}$.

Taper-minus. All 12 internal normalizers are TaperNorm, following the operator and calibration of Kanavalau et al. (2026) (their Section 3 and Appendix C), without their auxiliary scale loss. For an input $h \in \mathbb{R}^d$ at global update u , with gains γ and $\tilde{\gamma}$,

$$r(h) = \sqrt{\|h\|_2^2/d + 10^{-6}}, \quad y = g(u) \frac{h}{r(h)} \odot \gamma + (1 - g(u)) c h \odot \tilde{\gamma}. \quad (3)$$

The scalar c of each site is calibrated during updates $u = 1$ –763 as the ratio of bias-corrected exponential moving averages (one EMA step per optimizer update, new-sample weight 0.01) of $\|h \odot \gamma\|^2/r(h)$ and $\|h \odot \gamma\|^2$. These are collected from the pre-step forward passes of every microbatch. After update $u = 763$, c is frozen and $\tilde{\gamma}$ is initialized from γ ; $\tilde{\gamma}$ first receives an optimizer step at $u = 764$ and stays trainable. The gate is $g(u) = 1$ for $u \leq 763$, $g(u) = \frac{1}{2}[1 + \cos(\pi(u - 763)/5341)]$ for $763 < u < 6104$, and $g(u) = 0$ for $u \geq 6104$. At $g = 0$ no per-token statistic is computed and γ receives no gradient. Until $u = 763$ the two conditions are mathematically identical. The gate timing, GELU, learned positions, weight decay, learning-rate schedule and domain branching are choices of this study, so this is an adaptation of TaperNorm, not a replication of its experiments.

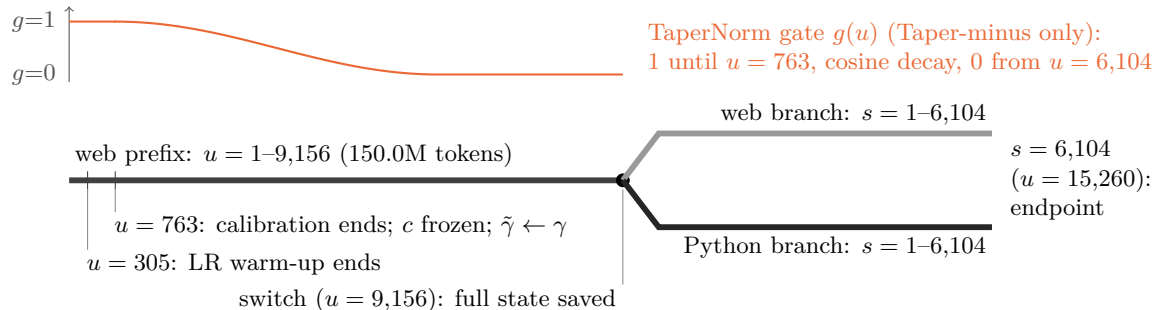


Figure 1: One condition and seed of the design (u : global update; s : continuation update). Both conditions start from identical initial weights and read the same data order for each seed. At the switch ($u = 9,156$), each condition’s complete state (weights, optimizer moments, loss scale and random-number state) is saved, and both of its branches restore that identical state. The web branch continues on C4; the Python branch trains on Python files from The Stack. The global learning-rate clock continues through the branches without a restart. Evaluations follow the schedule in Appendix A.

3.3 Training protocol

Optimization. The optimizer is AdamW (Loshchilov and Hutter, 2019) with $\beta = (0.9, 0.95)$ and $\epsilon = 10^{-8}$. All two-dimensional parameters, embeddings included, get weight decay 0.1; gain vectors get none. Gradients are clipped once per update to global norm 1.0, after accumulation. Each update uses $32 \times 512 = 16,384$ supervised tokens. The learning rate rises linearly to 6×10^{-4} over the first 305 updates and then follows a cosine decay to 6×10^{-5} at $u = 15,260$. The branches continue this global clock, with no learning-rate restart, optimizer reset, replay or regularization toward the prefix (Figure 1).

Pairing. Seeds 101, 102 and 103 are paired across conditions. A canonical RMS initialization is generated from the seed and explicitly copied into the Taper-minus model; $\tilde{\gamma}$ starts at 1 and is inactive until $u = 764$. Both conditions read the same pre-generated window order. Each run consists of a prefix of 9,156 updates (150,011,904 tokens) followed by two branches of 6,104 updates each (100,007,936 tokens). The web branch reads further unseen web windows and the Python branch reads Python windows; no window repeats within a trajectory. The primary allocation is 2.10B supervised tokens.

Precision. Training ran in FP16 autocast with dynamic loss scaling (Micikevicius et al., 2018). Master weights, optimizer moments, normalization and calibration statistics, and the loss were kept in FP32. A step whose gradients were nonfinite was to be retried on the same batch at a halved loss scale, so no update is ever skipped. This replaced the protocol’s original BF16 plan, because the available hardware (NVIDIA T4) lacks efficient BF16 support (Table 2).

3.4 Data

Sources and splits. Web text comes from C4 English (Raffel et al., 2020). Training uses the official training split, and the official validation split is hashed 50/50 into development and reserved-test sets. Code comes from the Python subset of The Stack (deduplicated) (Kocetkov et al., 2022). The dataset revisions are 1588ec454efa (C4) and 17cad72c886a (The Stack). Python files are grouped into connected components over their repository-name aliases (140,195 groups). Each group is assigned to a split by the value of $\text{sha256}(\text{group}) \bmod 100$: 0–89 train, 90–94 development

and 95–99 test. Empty documents, and Python files without a usable repository name, were dropped.

Deduplication. Documents are deduplicated exactly by content hash across all domains and splits. An approximate near-duplicate search uses MinHash (Broder, 1997) over GPT-2 token 5-grams (128 permutations, LSH with 32 bands of 4 rows), confirmed by exact Jaccard similarity ≥ 0.85 . When duplicates straddle splits, the copy in the higher-priority split is kept (test over development over training). This removed 0 exact and 2,182 near duplicates. Documents shorter than five tokens have no 5-gram and are covered only by exact deduplication (20 documents).

Recall of the near-duplicate search. With 32 bands of 4 rows, a pair with Jaccard similarity J becomes an LSH candidate with probability $1 - (1 - J^4)^{32}$. A pair at $J = 0.85$ is therefore missed with probability 5.5×10^{-11} , and a pair at $J = 0.70$ is still found with probability 0.9998. This analytic recall is the evidence that near duplicates above the threshold were found. The preparation code also compared a deterministic random sample of 2,043 cross-split document pairs by exact Jaccard, and none reached 0.85. Because two randomly chosen documents are almost never near duplicates, that comparison is a sanity check with negligible power to detect leakage, not evidence of its absence.

Packing and quotas. Text is tokenized with GPT-2 BPE, with one EOS appended per document, and packed into 513-token windows at stride 512 with no padding. Windows are packed across document boundaries without an attention mask: a window can hold the end of one document, an EOS token and the start of the next, and attention can cross the EOS. Positions restart at 0 in every window. Each label is attributed to its own document (a label after an EOS belongs to the new document), so per-document sums reconstruct the corpus CE exactly. Quotas are 260,000,257 web and 110,000,129 Python training tokens, and 2,097,152 development labels (4,096 windows) per domain. The *quick* development set is the first 262,144 of those labels. Reserved-test sets of 8,388,608 labels per domain were sealed and never used. The non-whitespace code CE used to measure adaptation excludes class W only.

3.5 Evaluation schedule and decision rules

Full development evaluations ran at 13 prefix points and at 25 continuation points per branch. Quick development evaluations ran at the switch and at 80 continuation points per branch, dense early on (Appendix A). Every full evaluation stores per-document and per-class sufficient statistics.

The secondary *matched-adaptation* analysis is fixed in advance. It fixes targets at the RMS Python branch’s quick-dev code CE at $s \in \{1525, 3050, 6104\}$. It finds the first downward crossing of each target by Taper-minus within an observed bracket of at most 100 updates (no extrapolation) and interpolates Taper-minus’s web CE at that crossing. It then compares the two models’ web forgetting relative to their own $s = 0$.

The decision rules in Table 1 are applied in order once all three seed groups are complete. They are screening criteria, not significance tests. With three seeds, the reported 95% interval $\bar{D} \pm 4.303 s_D / \sqrt{3}$ is descriptive.

Choice of thresholds. The protocol fixed the thresholds of 0.015 and 0.03 nats/token in advance as investment thresholds for a screening pilot. It records no power analysis or formal derivation for them, and we do not supply one after the fact. For orientation, a cross-entropy difference of 0.015 or

Table 1: Pre-specified decision rules, applied in order. CE values are in nats/token; a bar denotes the mean over seeds; s is the continuation update.

#	Category	Condition
1	Invalid or incomplete	Missing seed group, unresolved correctness or data failure, nonfinite primary run, or resource termination
2	Comparability / adaptation limited	Any seed’s relative prefix web-CE gap $> 2\%$, or either condition’s Python branch improves non-W code CE by < 0.05
3	Opposite direction	$\bar{D}(6104) \leq -0.03$
4	Proceed to design next study	$\bar{D}(6104) \geq 0.03$; $D(6104) > 0$ in every seed; $\bar{D}(3050) \geq 0.015$; $\bar{Q} > 0.5 \bar{D}$; mean matched differential forgetting > 0 at the furthest common target
5	Transient only	$\bar{D}(6104) < 0.015$ and quick-dev $\bar{D}(s) \geq 0.05$ at some scheduled $s \leq 1000$
6	Stop: small observed effect	$\bar{D}(6104) < 0.015$
7	Inconclusive	Anything else

0.03 nats/token corresponds to a 1.5% or 3.0% change in perplexity ($e^{0.015} \approx 1.015$, $e^{0.03} \approx 1.030$). A mean between the two thresholds falls into the inconclusive category rather than into either decision.

3.6 Pre-specification, timeline and amendments

Protocol. The hypothesis, design, estimands, evaluation schedule and decision thresholds come from a written protocol (“protocol v3”), drafted on 12 September 2026, before the corpus was prepared or any model was trained on it. The protocol text is included in the project repository. It was not lodged with an external registry, which is a limitation (Section 7). The amendments in Table 2 were made before any primary-run update; none of them changes the model, data, token budgets, seeds, endpoint or thresholds.

Timeline of the primary run (3 October 2026, UTC).

- 12:18: the run starts.
- 12:19:23: the run log records the frozen configuration hash (c18441799eee2b6a...). The hashed configuration contains the decision thresholds, and the hash can be recomputed from the released `config.json` by the runner’s own procedure.¹
- 14:25–15:26: seed 101’s four endpoint cross-entropies (both conditions, both branches) appear in the run’s live log. Seed 101’s D could have been computed from them.
- 16:07: commit 3ee1dc4 is made, containing the protocol text and the runner script.
- 21:40: the run ends, and the same process applies the decision rules and writes the decision report.

¹The hash is *not* the SHA-256 of the file itself; `sha256sum config.json` gives a different value. To reproduce it, load the JSON, remove the keys `config_sha256` and `checkpoint_seconds`, serialize the rest as canonical JSON (keys sorted, separators `"`, `"` and `:"`, no whitespace) and take the SHA-256 of the UTF-8 bytes. `paper/README.md` gives a one-line command, and `paper/make_assets.py` asserts the match on every build.

Table 2: Protocol amendments, all made before the primary run.

Date	Amendment
2026-09-28	NVIDIA T4 hardware: FP16 autocast with dynamic loss scaling replaces BF16; FP32 master weights, moments, normalization and EMA statistics, and loss. Results are not pooled with any BF16 run.
2026-09-30	A cap of 50 free cloud GPU-hours, spread over sessions and weekly quota resets, replaces the original 24-GPU-hour paid budget; the scientific allocation is unchanged.
2026-10-01	RMS and Taper-minus are trained as independent processes on two GPUs, with no shared state.
2026-10-03	Microbatch of 8 sequences $\times$ 4 accumulation steps (the protocol’s default layout; effective batch unchanged). For storage, model weights are kept only for the switch state and branch endpoints; the per-document and per-class statistics required at every full-development point are kept.

Because the commit came after seed 101’s endpoint values had been printed, the commit on its own does not establish pre-specification, and we disclose that here and in Section 7. The evidence that the analysis was fixed in advance is different. The thresholds used by the run were bound into the configuration hash logged at 12:19:23, before any outcome existed. The runner script, whose decision thresholds appear on line 82, has the same SHA-256 (66769e7de30ec36f...) in every commit of the repository history, and that hash matches the one recorded by the run itself. The decision report was produced by that same process.

Engineering runs. Before the primary run, engineering runs measured throughput and verified checkpoint resume. All of them were discarded:

- local development runs on stand-in corpora (WikiText-103 and CodeSearchNet functions), which trained RMS models only;
- preflights of an earlier execution pipeline under engineering seed 997, which timed each training phase by repositioning the update clock rather than by training a prefix to the switch;
- a smoke test of about 20 minutes on the final runner, which repeated the first 2,197 (Taper-minus) and 2,336 (RMS) prefix updates of seed 101.

None of them produced a Taper-minus-versus-RMS continuation contrast on the C4 and Stack data, so no value of D was computed before the primary run.

4 Execution and implementation checks

The primary run took one 9.36-hour session on a cloud machine with two NVIDIA Tesla T4 GPUs (PyTorch 2.11.0, CUDA 12.8), with RMS on one GPU and Taper-minus on the other. All six condition–seed runs completed all 21,364 updates. There were no nonfinite losses, gradients or weights; the loss scale grew to 16,384 and never needed a retry (0 retries in total). GPU kernels used for the backward pass are nondeterministic, so results are reproducible to within numerical noise, not bitwise.

Gradient clipping. Clipping at global norm 1.0 is part of the same optimization recipe for both conditions; we report how often it was active, per phase, from the per-update training logs

Table 3: Share of updates with gradient clipping active (pre-clipping global norm > 1.0), mean over seeds with the range across seeds in parentheses, by training phase. u is the global update.

Phase	Updates	RMS	Taper-minus
Prefix, $u \leq 763$ (calibration)	763	15.6% (15.3–16.0)	15.6% (15.3–16.0)
Prefix, $764 \leq u \leq 6,103$ (gate decaying)	5,340	4.3% (1.8–6.4)	10.9% (6.5–17.2)
Prefix, $6,104 \leq u \leq 9,156$ (gate zero)	3,053	15.4% (8.8–22.1)	49.9% (38.1–66.6)
Web branch	6,104	88.4% (83.1–93.1)	95.3% (92.5–98.4)
Python branch	6,104	95.1% (92.7–97.6)	95.9% (94.8–97.3)

(Table 3). During calibration, where the two conditions are mathematically identical, clipping was active in 16% of updates for both, as it should be. While the gate decayed it was less frequent (4% of updates for RMS, 11% for Taper-minus, as means over seeds). After the gate reached zero it became more frequent, more so for Taper-minus (50% vs 15% of the remaining prefix updates). On both continuation branches it was active in 88–96% of updates in both conditions. We report these rates descriptively and did not analyse their consequences.

Implementation checks. The training code is a single script that transcribes a separately tested reference implementation of the model, optimizer step and analysis. Before the run, it was checked against that reference in seven ways:

- (i) identical learning-rate and gate values at all 15,260 updates;
- (ii) identical paired initial tensors;
- (iii) FP32 training updates during the calibration phase that match the reference update to a relative tolerance of 2×10^{-5} ;
- (iv) bit-identical FP32 model outputs at updates spanning every TaperNorm gate state;
- (v) a vectorized evaluator that reproduces the reference evaluator’s CE and class sums on the real development data, to a relative tolerance of 10^{-9} ;
- (vi) identical decision categories on 3,000 randomly generated evidence sets;
- (vii) checkpoint resume whose deviation from an uninterrupted run is of the same order as run-to-run GPU nondeterminism.

The primary contrasts in Section 5 were also recomputed independently from the raw evaluation records, and they agree with the frozen analysis output.

5 Results

5.1 Validity checks

Table 4 shows that every guardrail passed.

- **Comparable prefixes.** The relative gap in prefix web CE was 0.28–0.34% (limit 2%), with Taper-minus slightly lower in every seed. Over the last five prefix evaluations, both conditions were still improving at nearly the same rate (mean slope -0.00244 vs. -0.00249 nats per million tokens).

Table 4: Validity checks and actual forgetting F (full-dev web CE, nats/token). “Gap” is the absolute prefix web-CE difference relative to RMS (limit 2%). “Code gain” is the Python branch’s reduction in non-W code CE from the switch to $s = 6,104$ (minimum 0.05).

Seed	Prefix web CE			F , web branch		F , Python branch		Code gain (R / T)
	RMS	Taper	Gap	RMS	Taper	RMS	Taper	
101	4.5846	4.5716	0.28%	−0.128	−0.128	+1.655	+1.636	3.55 / 3.55
102	4.5795	4.5666	0.28%	−0.123	−0.122	+1.688	+1.668	3.60 / 3.46
103	4.5931	4.5773	0.34%	−0.128	−0.124	+1.655	+1.633	3.35 / 3.34

Table 5: Primary contrast D at $s = 6,104$ and secondary contrasts (nats/token; full development set). Positive values mean more excess web forgetting for Taper-minus. “Matched” is the matched-adaptation differential forgetting at the furthest target reached by every seed ($s = 6,104$).

Seed	D	$D(3050)$	$D(1525)$	G_{web}	Q	Matched ($s=6104$)
101	−0.0200	+0.0109	+0.0369	−0.0006	−0.0194	−0.0308
102	−0.0207	−0.0124	+0.0224	−0.0009	−0.0199	−0.0182
103	−0.0262	+0.0193	−0.0286	−0.0042	−0.0220	−0.0286
Mean	−0.0223	+0.0059	+0.0102	−0.0019	−0.0204	−0.0259
SD	0.0034	0.0165	0.0344	0.0020	0.0014	0.0067

- **Strong adaptation.** Both conditions adapted strongly to Python: non-W code CE fell by 3.34–3.60 nats (minimum 0.05), and the adaptation curves almost coincide (Figure 2c).
- **Large forgetting in both conditions.** The Python branch raised web CE by 1.63 to 1.69 nats/token relative to the switch, while web continuation lowered it by 0.12–0.13 (Figure 2a).

5.2 Primary endpoint

Table 5 gives the primary results. Mean $D = -0.0223$ nats/token (SD 0.0034; descriptive 95% interval $[-0.0308, -0.0139]$), and D is negative in all three seeds. Since $\bar{D} < 0.015$ and no early transient reached 0.05 (Section 5.3), the pre-specified decision is **stop: small observed effect** (rule 6). The data provide no evidence for H1.

The other pre-specified quantities point the same way as D :

- G_{web} is close to zero (mean −0.0019), so the difference sits in the Python branch itself: the direct differential forgetting Q averages −0.0204.
- The matched-adaptation contrast at $s = 6,104$ averages −0.0259, negative in every seed. On the quick development set, Taper-minus reached the RMS code-CE target before the corresponding RMS update in 9 of 9 seed–target pairs (Appendix B). At the furthest target, then, Taper-minus also showed slightly less web forgetting when compared at matched code adaptation.
- The per-document and per-class decompositions reconstruct D exactly. The effect is broad rather than driven by a few documents: median per-document D_i lies between −0.027 and −0.023. By token class, alphanumeric tokens contribute negatively in all 3 seeds and give the largest share in 2 of them; in seed 101, punctuation and symbol tokens contribute more (Appendix B).

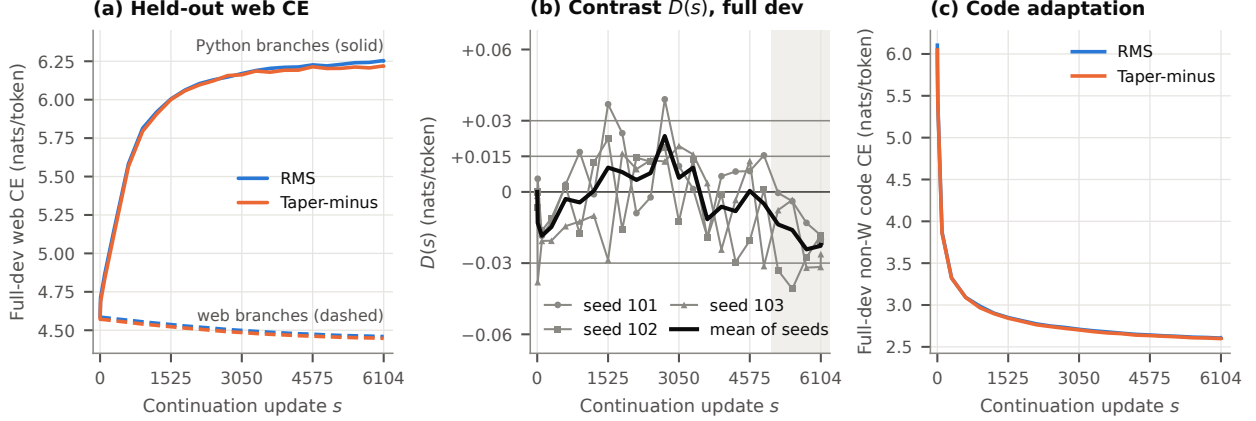


Figure 2: Continuation trajectories on the full development sets (means over three seeds unless stated; s is the continuation update). **(a)** Held-out web CE in all four branches. RMS and Taper-minus nearly coincide within each branch, and the Python branches show large forgetting. **(b)** The contrast $D(s)$ for each seed and their mean. The labelled horizontal lines are the pre-specified thresholds (+0.03 proceed, +0.015 small-effect bound, -0.03 opposite direction); the decision uses only $s = 6,104$ (and $s = 3,050$ for rule 4). The shaded band marks the late window ($s \geq 5,185$) used in the post hoc description of Section 5.3. **(c)** Non-whitespace code CE on the Python branch, showing near-identical adaptation in the two conditions.

- Rare tokens (flag R, 5,147–5,285 labels per seed) have a more negative per-label contrast (mean D on R between -0.206 and -0.082), but because they are rare they contribute only -0.0005 to -0.0002 to D , at most 3% of it (Appendix B).

5.3 How the contrast developed during continuation

The difference between the conditions is small in absolute terms: $|\bar{D}|$ is 1.3% of the mean Python-branch forgetting, which is 1.66 nats/token. The decision uses only the endpoint, but the full-development contrast was measured at 25 points (Figure 2b). The windows below group those points after the fact; they describe the trajectory and were not part of the protocol.

- **Early** ($s = 10$ –305): the seed-mean $D(s)$ was between -0.018 and -0.013 .
- **Middle** ($s = 610$ –4,880, 15 points): the seed-mean $D(s)$ averaged $+0.002$ and ranged from -0.012 to $+0.024$, with no consistent sign.
- **Late** ($s = 5,185$ –6,104, 5 points): the seed-mean $D(s)$ was negative at all 5 points and averaged -0.020 . Throughout this window the learning rate was within 9% of its final value.

In the late window $D(s)$ was negative in every seed at every point (15 of 15 seed–point values), and the across-seed SD fell from an average of 0.0164 over the middle window to 0.0034 at the endpoint, 4.8 times smaller. Seed agreement was not unique to the end, however: at $s = 100$ the seed-mean $D(s)$ was -0.0185 with an across-seed SD of 0.0021. The trajectory therefore shows two seed-consistent negative phases, early and late, separated by a noisy middle, and low seed spread is not specific to the low-learning-rate phase. The negative endpoint reflects the late phase, not a difference present throughout continuation. The pre-specified transient check was not triggered: the quick-dev seed-mean $D(s)$ peaked at $+0.015$ (at $s = 400$) within the first 1,000 updates, against a threshold of 0.05.

Table 6: Exploratory: scale of the activations entering the 12 internal normalizers, from the logged diagnostic probes, as natural-log ratios of per-site scales. Rows marked $|\cdot|$ take the absolute log-ratio for each of the 36 site-seed pairs and then the mean; signed rows are plain means, with the corresponding multiplicative factor in parentheses.

Natural-log scale ratio (mean over 12 sites $\times$ 3 seeds)	RMS	Taper-minus
Code vs. web at the switch, $ \cdot $	0.097	0.123
Web-probe change, web branch, $ \cdot $	0.206	0.054
Web-probe change, Python branch, $ \cdot $	0.207	0.077
signed, web branch (factor)	$-0.206 (\times 0.81)$	$+0.040 (\times 1.04)$
signed, Python branch (factor)	$-0.196 (\times 0.82)$	$+0.060 (\times 1.06)$
site-seed pairs that shrank, web / Python branch	36 / 30 of 36	6 / 3 of 36
Code-specific change (Python minus web branch), $ \cdot $	0.042	0.042

5.4 Exploratory analyses (not pre-specified)

The analyses in this subsection were chosen after the results were known. They are descriptive, and none of them enters the decision.

Activation scales at the normalizer inputs. The run logged forward-only probes of the activation h entering each of the 12 internal normalizers, on the last 256 training windows of each domain (131,072 positions per domain), at the switch and at the end of each branch. For each site we take the scale to be the root mean square of $\|h\|$ over the probe positions and work with natural-log ratios of scales, which treat growth and shrinkage symmetrically. For the absolute quantities ($|\cdot|$) we take the absolute log-ratio for each of the 36 site-seed pairs and then average; the signed quantities are plain averages, also shown as multiplicative factors. Table 6 gives three quantities:

- **The code-vs-web gap at the switch.** The mean absolute log-ratio was 0.097 (RMS) and 0.123 (Taper-minus): code and web activations differed in scale by roughly 10% and 13%.
- **The change in the web-probe scale during each branch.** In the RMS model the scale shrank, by a factor of 0.81 in the web branch (36 of 36 site-seed pairs shrank) and 0.82 in the Python branch (30 of 36). In Taper-minus it changed much less and mostly grew, by factors of 1.04 and 1.06 (mean absolute log-ratios 0.054 and 0.077).
- **The code-specific part of that change** (Python branch minus web branch, per site and seed): a mean absolute log-ratio of 0.042 (RMS) and 0.042 (Taper-minus), about 4% in both.

The largest scale change, the shrinkage in the RMS model, happened in both branches, so it was not caused by the code data. The code-specific change was small and the same size in both architectures.

Document bootstrap. To see how much of the uncertainty in D comes from the finite development set, we resampled the 4,409 web development documents with replacement (10,000 replicates; the same resampled documents for all arms and seeds) and recomputed the endpoint D . The 95% percentile interval for mean D is $[-0.0235, -0.0211]$, with a bootstrap standard error of 0.0006; per-seed standard errors are 0.0009–0.0010 (Appendix B). This interval treats the trained models as fixed, so it measures only evaluation-sample noise. Comparing like with like, the between-seed SD of D (0.0034) is about 3.6 times the per-seed bootstrap standard error, and the seed-based standard error of the mean (0.0020) is about 3.3 times the bootstrap standard error of the mean.

Uncertainty about D is therefore dominated by training variation across seeds, and the bootstrap interval must not be read as uncertainty about the effect itself.

6 Discussion

What the result shows. In this setting, replacing internal RMSNorm with TaperNorm did not increase persistent forgetting of web text after a switch to Python. The pre-specified screen for a meaningful excess ($\bar{D} \geq 0.015$) was not met, and every seed’s endpoint D was below zero. Under the protocol’s rules this direction is stopped. No seeds were added, no endpoint was changed and no follow-up was launched.

What it does not show. *Stop* is not evidence of equivalence (Lakens, 2017), and the negative endpoint is not evidence that TaperNorm protects against forgetting. That effect would have needed $\bar{D} \leq -0.03$, which was not reached. Apart from a brief early dip, the negative values appeared in the final, low-learning-rate phase, and the training regime, including learning-rate decay, is known to affect forgetting (Mirzadeh et al., 2020). Whether TaperNorm interacts with the end of the schedule is an open question that this pilot was not designed to answer. Seed-to-seed variation in neural-network training can be large (Dodge et al., 2020), and three seeds estimate it poorly. The descriptive interval is conditional on one prepared corpus sample and one training recipe; it is not a population bound.

Why the feared mechanism may have had little to act on. H1 rests on a story: training on code changes activation magnitudes, RMSNorm absorbs the change, and a fixed TaperNorm scale cannot. The exploratory scale analysis (Section 5.4) suggests that the starting condition of this story was only weakly present in our models:

- At the switch, code and web activations differed in scale by only about 10–13% on average (mean absolute log-ratios 0.097 and 0.123).
- The largest scale change during continuation, the shrinkage in the RMS model, occurred to a similar extent when training simply continued on web text (factors 0.81 and 0.82), so it was not a response to code.
- The code-specific scale change was about 4% per site in both architectures (mean absolute log-ratio 0.042).

A plausible reason for the RMS shrinkage is that RMSNorm makes each sub-layer’s output invariant to the scale of the residual stream it reads, so weight decay can shrink the weights that set that scale without changing the function (van Laarhoven, 2017; Zhang et al., 2019a). Once its gate is zero, Taper-minus’s internal layers depend on that scale, which would oppose the same shrinkage. We did not test this explanation. Two further features of the setting may also matter: Taper-minus keeps the final RMSNorm, which Kanavalau et al. (2026) identify as the essential scale anchor, and $\tilde{\gamma}$ and all weight matrices remain trainable after the taper.

These observations carry four caveats. They are exploratory, not pre-specified. The probes use the last 256 training windows of each domain, which the models may have seen during training. They describe mean scales over tokens, whereas RMSNorm also normalizes variation between individual tokens. And they are descriptive, not causal: they show that the premise was weak, not that this is why there was no effect.

7 Limitations

- **Scale.** The models have 17,718,784 parameters, about 73% of them in embeddings, and see 150M prefix tokens. Embedding drift may mediate or interact with the condition effect, and results may not transfer to larger models or longer training.
- **Breadth.** There is one domain pair (web to Python), one data sample, one optimizer recipe and one hardware and precision setting (FP16 on T4, not the protocol’s original BF16). The auxiliary-loss variant of TaperNorm (Taper-plus) was not run.
- **Statistics.** Three paired seeds share one prepared corpus, so uncertainty is conditional on that corpus, and the interval in Table 5 is descriptive. All measurements use development data; the reserved test sets remain sealed for any confirmatory study.
- **Pre-specification.** The protocol was kept privately and not registered externally. The earliest public record of the protocol and analysis code, a git commit, postdates the start of the primary run, and it also postdates the appearance of seed 101’s endpoint values in the run’s live log (Section 3.6). Git commit dates are set by the author’s machine and are not independent timestamps. The evidence that the analysis was fixed in advance is the configuration hash logged before any outcome and the unchanged runner hash, not the commit date.
- **Thresholds.** The decision thresholds were set as investment thresholds without a power analysis.
- **Data audit.** The near-duplicate search is approximate. Its analytic recall is high for pairs above the similarity threshold; the comparison of 2,043 random cross-split document pairs is only a sanity check with negligible power; and repository names are an incomplete proxy for forks.
- **Secondary and exploratory analyses.** Matched adaptation interpolates between evaluated checkpoints and describes curves; it does not evaluate an actual model. The trajectory windows, the scale analysis and the document bootstrap were chosen after the results were known. Attention-entropy and gain diagnostics were logged but are not analysed here.

8 Conclusion

A small pilot, specified before training, found no evidence that TaperNorm increases persistent forgetting after a web-to-code domain shift: mean $D = -0.0223$ nats/token, against a pre-specified $+0.015$ bound. After a brief early dip, the difference stayed near zero until the final, low-learning-rate phase, when it turned negative; it was small, so we draw no conclusion about protection. Exploratory analyses suggest that the mechanism behind the concern had little to act on: the code-specific change in activation scale was small and similar in both architectures. Testing H1 more broadly would need a new pre-specified study with larger models, more seeds and more than one domain shift, ideally one that measures the scale premise directly. On its own, this pilot does not justify that study.

Code and data availability

The following are available in the project repository:

<https://github.com/DanielDotComCoder/domain-shift-forgetting>

- the code for data preparation, training, evaluation and the frozen analysis;
- the frozen configuration, with data and source hashes, and the decision report;
- every evaluation record (per-document and per-class sufficient statistics, and the diagnostic probes);
- the per-update training logs.

The script `paper/make_assets.py` regenerates every number, table and figure in this paper from those records. Three things are not in the repository:

- the model checkpoints (switch states and branch-final weights, about 2 GB);
- the prepared token arrays, which can be regenerated from the pinned dataset revisions with the released preparation code;
- any corpus text. C4 and The Stack are subject to their own licenses and terms of use.

Use of AI assistance

AI systems, including Claude (Anthropic), were used extensively in this project. They helped draft and revise the study protocol; wrote the data-preparation, training, orchestration and analysis code and its tests; operated the cloud training runs through the Kaggle API; and drafted this manuscript, including the script that generates its numbers, tables and figures. The author directed the project, approved the study design, every protocol amendment and the decision to stop, and takes full responsibility for the content of this paper, including any errors.

References

- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Andrew Brock, Soham De, Samuel L. Smith, and Karen Simonyan. High-performance large-scale image recognition without normalization. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.
- Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of SEQUENCES 1997*, pages 21–29, 1997.
- Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*, 2020.
- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.
- Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4): 128–135, 1999.
- Kshitij Gupta, Benjamin Thérien, Adam Ibrahim, Mats L. Richter, Quentin Anthony, Eugene Belilovsky, Irina Rish, and Timothée Lesort. Continual pre-training of large language models: How to (re)warm your model? *arXiv preprint arXiv:2308.04014*, 2023.
- Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360, 2020.
- Bobby He and Thomas Hofmann. Simplifying transformer blocks. In *International Conference on Learning Representations*, 2024.

- Adam Ibrahim, Benjamin Thérien, Kshitij Gupta, Mats L. Richter, Quentin Anthony, Timothée Lesort, Eugene Belilovsky, and Irina Rish. Simple and scalable strategies to continually pre-train large language models. *Transactions on Machine Learning Research*, 2024.
- Andrei Kanavalau, Carmen Amo Alonso, and Sanjay Lall. Gated removal of normalization in transformers enables stable training and efficient inference. *arXiv preprint arXiv:2602.10408v1*, 2026. Version 1. Later versions of this arXiv entry carry a different title; this paper follows version 1.
- Andrej Karpathy. nanoGPT. <https://github.com/karpathy/nanoGPT>, 2022.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.
- Daniël Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Clare Lyle, Zeyu Zheng, Khimya Khetarpal, James Martens, Hado van Hasselt, Razvan Pascanu, and Will Dabney. Normalization and effective learning rates in reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.
- Seyed Iman Mirzadeh, Mehrdad Farajtabar, Razvan Pascanu, and Hassan Ghasemzadeh. Understanding the role of training regimes in continual learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- Quang Pham, Chenghao Liu, and Steven C. H. Hoi. Continual normalization: Rethinking batch normalization for online continual learning. In *International Conference on Learning Representations*, 2022.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- Vinay Venkatesh Ramasesh, Aitor Lewkowycz, and Ethan Dyer. Effect of scale on catastrophic forgetting in neural networks. In *International Conference on Learning Representations*, 2022.
- Twan van Laarhoven. L2 regularization versus batch and weight normalization. *arXiv preprint arXiv:1706.05350*, 2017.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

- Guodong Zhang, Chaoqi Wang, Bowen Xu, and Roger Grosse. Three mechanisms of weight decay regularization. In *International Conference on Learning Representations*, 2019a.
- Hongyi Zhang, Yann N. Dauphin, and Tengyu Ma. Fixup initialization: Residual learning without normalization. In *International Conference on Learning Representations*, 2019b.
- Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

A Evaluation schedule

Full development evaluations (2,097,152 labels per domain). Prefix: global updates $u = 763, 3052, 6104, 6409, 6714, 7019, 7324, 7629, 7934, 8239, 8544, 8849$ and 9156 . Continuation (each branch): $s \in \{0, 1, 10, 100\}$, every 305 updates from $s = 305$ to 6100 , and $s = 6104$.

Quick development evaluations (262,144 labels per domain). At the switch, and at continuation $s \in \{0, 1, 2, 5, 10, 20\}$, every 50 updates from $s = 50$ to 1000 , every 100 updates from $s = 1100$ to 6100 , and $s \in \{1525, 3050, 6104\}$ (80 points). Quick-dev values describe transients and feed matched adaptation. They never enter the full-dev primary contrast.

Diagnostics. Forward-only activation, gain and attention-entropy probes on the last 256 training windows of each domain, at the switch and at $s \in \{0, 10, 100, 1525, 6104\}$. The activation scales are analysed in Section 5.4 (exploratory); the other diagnostics are logged but not analysed here.

B Additional results

Table 7: Matched-adaptation differential forgetting (nats/token) and the interpolated Taper-minus continuation update s at which it reached each RMS code-CE target, with the observed bracket in parentheses.

Seed	Matched differential forgetting			Taper update at match (bracket)		
	$s=1525$	$s=3050$	$s=6104$	$s=1525$	$s=3050$	$s=6104$
101	−0.0075	−0.0038	−0.0308	1447 (1400–1500)	2740 (2700–2800)	5198 (5100–5200)
102	+0.0094	−0.0106	−0.0182	1475 (1400–1500)	2776 (2700–2800)	5775 (5700–5800)
103	−0.0259	+0.0145	−0.0286	1519 (1500–1525)	3024 (3000–3050)	5862 (5800–5900)

Table 8: Decomposition of the endpoint D (nats/token) into disjoint token classes (each class’s contribution weighted by its label share; the four contributions sum to D), and summaries of per-document effects D_i over the 4,409 web development documents. “Top-1% sum” is the signed contribution of the 1% of documents with the largest positive contributions.

Seed	Contribution to D by token class				Per-document D_i		
	W	A	P	X	Median	1%-trimmed	Top-1% sum
101	+0.0038	−0.0098	−0.0121	−0.0019	−0.0232	−0.0203	+0.0018
102	−0.0019	−0.0155	+0.0014	−0.0047	−0.0257	−0.0210	+0.0020
103	+0.0008	−0.0227	−0.0055	+0.0012	−0.0272	−0.0265	+0.0020
Labels	33,807	1,786,076	230,998	46,271	4,409 documents		

Table 9: Rare-token flag R (pre-specified; overlaps the four classes) and the exploratory document bootstrap of the endpoint D (10,000 replicates, documents resampled jointly across arms and seeds; conditional on the trained models).

Seed	R labels	Mean D on R	R contribution to D	Document-bootstrap 95% interval for D
101	5,267	-0.082	-0.0002	$[-0.0218, -0.0182]$
102	5,285	-0.206	-0.0005	$[-0.0226, -0.0189]$
103	5,147	-0.199	-0.0005	$[-0.0281, -0.0243]$
Mean of seeds				$[-0.0235, -0.0211]$